

The National Institute of Engineering
[An Autonomous Institution under VTU, Belagavi]
Mysore, Karnataka 570008, India



Department of Computer Science and Engineering

Course Name: **PYTHON PROGRAMMING**

Course Code: **1BPLC205B**

Semester: **2nd Semester**

Course Faculty: **Smitha B,**

Asst. Professor,

Dept. of CS&E, NIE, Mysore

Module 2.2

Tuples

Tuples

Tuples are used for grouping data:

We saw earlier that we could group together pairs of values by surrounding with parentheses. Recall this example:

```
>>> year_born = ("Paris Hilton", 1981)
```

This is an example of a **data structure** — a mechanism for grouping and organizing data to make it easier to use.

The pair is an example of a **tuple**. Generalizing this, a tuple can be used to group any number of items into a single compound value. Syntactically, a tuple is a comma-separated sequence of values. Although it is not necessary, it is conventional to enclose tuples in parentheses:

```
>>> julia = ("Julia", "Roberts", 1967, "Duplicity", 2009, "Actress",  
            ↪ "Atlanta, Georgia")
```

Tuples support the same sequence operations as strings. The index operator selects an element from a tuple.

```
>>> julia[2]
1967
```

But if we try to use item assignment to modify one of the elements of the tuple, we get an error:

```
>>> julia[0] = "X"
TypeError: 'tuple' object does not support item assignment
```

```
>>> julia = julia[:3] + ("Eat Pray Love", 2010) + julia[5:]
>>> julia
('Julia', 'Roberts', 1967, 'Eat Pray Love', 2010, 'Actress', 'Atlanta, _
↳ Georgia')
```

To create a tuple with a single element (but you're probably not likely to do that too often), we have to include the final comma, because without the final comma, Python treats the (5) below as an integer in parentheses:

```
>>> tup = (5,)
>>> type(tup)
<class 'tuple'>
>>> x = (5)
>>> type(x)
<class 'int'>
```

Tuple assignment

Once in a while, it is useful to swap the values of two variables. With conventional assignment statements, we have to use a temporary variable. For example, to swap `a` and `b`:

```
>>> temp = a
>>> a = b
>>> b = temp
```

If we have to do this often, this approach becomes cumbersome. Python provides a form of **tuple assignment** that solves this problem neatly:

```
>>> a, b = b, a
```

The left side is a tuple of variables; the right side is a tuple of values. Each value is assigned to its respective variable. All the expressions on the right side are evaluated before any of the assignments. This feature makes tuple assignment quite versatile.

Naturally, the number of variables on the left and the number of values on the right have to be the same:

```
>>> a, b, c, d = 1, 2, 3
ValueError: unpack tuple of wrong size
```

Tuples as return values

Functions can return tuples as return values. For example, we could write a function that swaps two parameters:

```
def swap(x, y):  
    return y, x
```

Then we can assign the return value to a tuple with two variables:

```
a, b = swap(a, b)
```

In this case, there is no great advantage in making `swap` a function. In fact, there is a danger in trying to encapsulate `swap`, which is the following tempting mistake:

```
def swap(x, y):          # incorrect version  
    x, y = y, x
```

If we call this function like this:

```
swap(a, b)
```

then `a` and `x` are aliases for the same value. Changing `x` inside `swap` makes `x` refer to a different value, but it has no effect on `a` in `__main__`. Similarly, changing `y` has no effect on `b`.

This function runs without producing an error message, but it doesn't do what we intended. This is an example of a semantic error.

Composability of Data Structures

We saw in an earlier chapter that we could make a list of pairs, and we had an example where one of the items in the tuple was itself a list:

```
students = [  
    ("John", ["CompSci", "Physics"]),  
    ("Vusi", ["Maths", "CompSci", "Stats"]),  
    ("Jess", ["CompSci", "Accounting", "Economics", "Management"]),  
    ("Sarah", ["InfSys", "Accounting", "Economics", "CommLaw"]),  
    ("Zuki", ["Sociology", "Economics", "Law", "Stats", "Music"])]
```

Tuples items can themselves be other tuples. For example, we could improve the information about our movie stars to hold the full date of birth rather than just the year, and we could have a list of some of her movies and dates that they were made, and so on:

```
julia_more_info = ( ("Julia", "Roberts"), (8, "October", 1967),  
    "Actress", ("Atlanta", "Georgia"),  
    [ ("Duplicity", 2009),  
      ("Notting Hill", 1999),  
      ("Pretty Woman", 1990),  
      ("Erin Brockovich", 2000),  
      ("Eat Pray Love", 2010),  
      ("Mona Lisa Smile", 2003),  
      ("Oceans Twelve", 2004) ])
```

Notice in this case that the tuple has just five elements — but each of those in turn can be another tuple, a list, a string, or any other kind of Python value. This property is known as being **heterogeneous**, meaning that it can be composed of elements of different types.

The Tuple Data Type

- The tuple data type is almost identical to the list data type
- Tuples are typed with parentheses - (and)
- Tuples are immutable - cannot have their values modified, appended, or removed



```
>>> eggs = ('hello', 42, 0.5)
>>> eggs[0]
'hello'
>>> eggs[1:3]
(42, 0.5)
>>> len(eggs)
3
```

```
>>> eggs = ('hello', 42, 0.5)
>>> eggs[1] = 99
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    eggs[1] = 99
TypeError: 'tuple' object does not support item assignment
```

Immutable list = tuple

```
>>> tup_nie = ('ISE', 'CSE', 'North Campus')
>>>
>>> tup_nie[0]
'ISE'
>>> tup_nie[0:2]
('ISE', 'CSE')
>>>
>>> len(tup_nie)
3
```

Tuple is created using a paranthesis '(' and ')' unlike square brackets [] which is used for list.

Items in a tuple cannot be changed (immutable)

```
>>> tup_nie = ('ISE', 'CSE', 'North Campus')
>>>
>>> tup_nie[2] = 'South Campus'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

Converting Types with the list() and tuple()

- The functions list() will convert the values passed to it into list type and tuple() will convert the values passed to it into tuple version respectively
- This is done to convert mutable to immutable and vice versa

```
>>> tuple(['cat', 'dog', 5])      # tuple(List)
('cat', 'dog', 5)
>>> list(('cat', 'dog', 5))      # list (tuple)
['cat', 'dog', 5]
>>> list('hello')                #list(string)
['h', 'e', 'l', 'l', 'o']
```

id function in python

id function returns the reference of the variable.

Immutable objects like string cannot be changed. This can be demonstrated with the help of a id()

```
>>> bacon = 'Hello'
>>> id(bacon)
140643522309936
>>>
>>> bacon += 'world'
>>> id(bacon)
140643524006320
>>> 
```

type function in python

```
>>> type(('hello',))  
<class 'tuple'>  
>>> type(('hello'))  
<class 'str'>
```

If you have only one value in your tuple, you can indicate this by placing a trailing comma after the value inside the parentheses.

Otherwise, Python will think you've just typed a value inside regular parentheses. The comma is what lets Python know this is a tuple value.

Guess the type

1. `type([1,2,3,4,5])`

Output: list

2. `type(1)`

Output: int

3. `type('hello')`

Output : str

4. `type(('hello'))`

Output: str

5. `type(('hello',))`

Output : tuple

6. `type(0.5)`

Output: float

7. `type({1,2,3,4,5,6,7})`

Output : set

8. `type({1:'hi',3:'hello',10:'pink'})`

Output : dict

Solution!!!

- The proper way to “mutate” a string is to use slicing and concatenation to build a new string by copying from parts of the old string.

```
>>> name = 'Zophie a cat'
>>> newName = name[0:7] + 'the' + name[8:12]
>>> name
'Zophie a cat'
>>> newName
'Zophie the cat'
```

Assignment 4

1. What is []?
2. How would you assign the value 'hello' as the third value in a list stored in a variable named spam? (Assume spam contains [2, 4, 6, 8, 10].)

For the following three questions, let's say spam contains the list ['a', 'b', 'c', 'd'].

3. What does spam[int('3' * 2) / 11] evaluate to?
4. What does spam[-1] evaluate to?
5. What does spam[:2] evaluate to?

For the following three questions, let's say bacon contains the list [3.14, 'cat', 11, 'cat', True].

6. What does bacon.index('cat') evaluate to?
7. What does bacon.append(99) make the list value in bacon look like?

8. What does `bacon.remove('cat')` make the list value in `bacon` look like?
9. What are the operators for list concatenation and list replication?
10. What is the difference between the `append()` and `insert()` list methods?
11. What are two ways to remove values from a list?
12. Name a few ways that list values are similar to string values.
13. What is the difference between lists and tuples?
14. How do you type the tuple value that has just the integer value 42 in it?
15. How can you get the tuple form of a list value? How can you get the list form of a tuple value?
16. Variables that “contain” list values don’t actually contain lists directly. What do they contain instead?
17. What is the difference between `copy.copy()` and `copy.deepcopy()`?